Task 1.1.3 | Finalisation

Final report: abstract and technical description (7 pages total)

Eunatia Ngobeni
92131229
DLBSEPCP01_E | Cloud Programming
17 October 2026

LIVE STATIC EXPLAINER | SEPARATE MULTI-REGION TERRAFORM PROTOTYPE

Portfolio Part 3 – Finalisation Abstract

Project links

- **AWS-hosted project website (static explainer):**

https://d2quyljtx3dqoe.cloudfront.net/

- **GitHub repository:**

https://github.com/EunatiaN1/multi-region-active-passive-failover

Project, outcome, and design

This portfolio project combines an interactive website about multi-region active-passive failover with infrastructure as code. The website explains the proposed AWS architecture and lets readers explore normal and simulated failure paths. Its controls are client-side illustrations: they do not invoke AWS APIs, change DNS, or demonstrate an actual recovery event.

The currently live website hosts that explainer as a static page. Its Terraform deployment provisions an S3 bucket with public access blocked, default encryption, versioning, and a lifecycle rule for non-current versions. CloudFront uses Origin Access Control (OAC) to read the object, redirects viewers to HTTPS, and applies the AWS-managed CachingOptimized cache policy. This deployed static-site root is deliberately independent of the new multi-region application example.

The submitted architecture proposes a primary application in eu-central-1 with a public Application Load Balancer across two Availability Zones and an EC2 Auto Scaling group. CloudFront forwards normal requests to the load balancer. A secondary S3 website in us-east-1 holds a maintenance page; CloudFront can request it when the primary origin returns HTTP 500, 502, 503, or 504. The original diagram also shows Route 53 health checks and DNS failover. These are separate mechanisms: CloudFront origin failover is response-code-driven, while DNS failover requires managed records and health checks for a controlled hostname.

The separate multi-region Terraform variant uses CloudFront's AWS-generated hostname so it can be inspected without a custom domain. It defines the primary VPC, ALB, EC2 Auto Scaling application, secondary S3 maintenance website, CloudFront origin group, and CloudWatch/SNS target-health alerting. It deliberately omits Route 53 records, DNS health checks, and ACM certificate provisioning. It is therefore an infrastructure-code prototype for origin failover, not a complete end-to-end DNS-failover service. It does not replace the existing live static website.

Making of, validation, and reflection

The Terraform application tier uses an Amazon Linux launch template and a small Python HTTP service managed by systemd. The threaded service handles concurrent requests, returns 404 for unknown paths, and runs as a dedicated unprivileged user with a restricted capability set and systemd filesystem protections. The target group checks `/health`; a distinct `/origin-health` CloudFront path is available for manual checking and does not drive Route 53. The Auto Scaling defaults are two minimum and desired instances, up to six, with target CPU utilization of 50 percent. Instance metadata requires IMDSv2 tokens. The ALB allows origin traffic from CloudFront's managed origin-facing prefix list, and instances accept HTTP only from the ALB security group.

The secondary website bucket uses a generated prefix and permits anonymous reads only for its maintenance `index.html`. The website endpoint itself is HTTP-only and directly reachable, so it must contain no confidential information. CloudFront can deliver a selected failover response through its HTTPS edge endpoint. The CloudWatch alarm observes unhealthy ALB targets and publishes to an SNS topic; an optional email recipient must confirm the subscription.

The project history records Terraform 1.12.2 formatting and provider-schema validation as passing, as well as eight automated, plan-only Terraform tests using a mocked AWS provider. The tests cover default capacity, invalid Auto Scaling capacity order, target health checks, IMDSv2, ALB and instance security-group boundaries, CloudFront failover response codes, maintenance-bucket public-access settings, alarm wiring, optional email subscription, project-name and region validation, and valid VPC subnet sizing. They require no AWS credentials and create no resources; they do not verify AWS permissions, deployment, or service behavior. Terraform was not available in the environment used to regenerate this submission, so the recorded checks were not rerun during this revision. The multi-region stack has not been applied or runtime-tested. The Phase 2 submission reports an earlier deployment and tests, but its corresponding source, state, and logs were not available to reproduce those results against this configuration. No live failover, scaling, alarm, or recovery result is claimed here.

Finalisation improved the application bootstrap, constrained network access, added input validation and mock-plan tests, and aligned this report with the configuration actually present. The remaining gaps are material: the multi-region stack is not deployed; CloudFront origin failover, alarms, scaling, and recovery have not been observed in AWS; and the Route 53 DNS layer is not implemented. Completing DNS failover requires a controlled domain and hosted zone, an explicit primary/standby DNS design, HTTPS certificate and endpoint decisions, tested health-check behavior, and account-specific cost and recovery review. A private or console S3 URL is not a domain or hosted-zone identifier. The validation register identifies the current evidence and the steps still needed. No tutor feedback was supplied for this revision; authentic feedback should be added if received.

Keywords: AWS, Terraform, Amazon S3, CloudFront, Application Load Balancer, Auto Scaling, active-passive, origin failover.

1. Assignment objective and success criteria

The assignment asks for a webpage hosted on AWS, a conception document and diagram, an implementation explanation, reproducible infrastructure as code, and a final abstract and product description. The design goals are availability, global access, adaptive compute, and repeatable provisioning. These are design goals, not measured service-level objectives.

The current live deliverable is a static interactive explainer. It introduces the components and illustrates normal and simulated failure flows. Its controls update the browser presentation only; they do not call AWS APIs, change DNS, or conduct a recovery test. Success for this live component means that the page is reachable and communicates its boundary clearly.

The multi-region Terraform directory is a separate prototype. It describes a primary application behind a multi-AZ load balancer, Auto Scaling, CloudFront origin-group failover, a secondary maintenance response, and notifications. It has not been deployed. Therefore the project does not claim operational multi-region availability, Route 53 failover, or a measured recovery time.

The success criteria should be evaluated at three evidence levels. Document review can establish that the code describes the intended resources and that the user-facing explanation distinguishes simulation from control. Mocked plan tests can check selected Terraform expressions and planned attributes. A controlled AWS exercise is needed to prove that deployed resources can be created, communicate, fail over, send alarms, and recover. Passing a lower evidence level does not imply that the higher levels have passed.

Availability is an end-to-end property. A CloudFront distribution can exist while its origin is unreachable; an ALB can be healthy while application data is unavailable; and an alarm can be configured without a confirmed notification path. A claimed service level would need a defined measurement window, monitored endpoint, outage definition, exclusions, and measured history. None is included in this portfolio's current evidence.

2. Proposed architecture and request paths

The intended normal path is visitor → DNS → edge delivery → primary-region application load balancer → healthy application instance. CloudFront is the global entry point and caches eligible responses. The ALB spans two Availability Zones in the primary region, and Auto Scaling maintains and adjusts application capacity.

The Terraform example implements a distinct origin-failure path: when the primary ALB origin returns HTTP 500, 502, 503, or 504, the CloudFront origin group can request the secondary S3 website origin. This is response-code failover at CloudFront. The S3 website is HTTP-only and directly reachable; its policy permits anonymous reads of the maintenance document only.

Route 53 DNS failover is depicted in the original concept but is not in the Terraform. DNS selection and CloudFront origin-group selection are separate mechanisms with different health signals and recovery behavior. The architecture figure in the portfolio is a conceptual drawing, not a deployed-resource inventory.

CloudFront origin-group failover applies to requests and response codes covered by its behavior and failover configuration. It is not a general regional control plane, does not test the origin with a separate health probe, and is not a promise that all user-visible errors cause a switch. An error body returned with HTTP 200 would not match the configured 5xx criteria. A transient selected 5xx can cause a maintenance response even if the regional service is recovering. The behavior, response caching, and origin timeouts must be considered in a live test.

DNS failover operates at name resolution. Resolver and client caching can delay observed transitions, and changing a DNS answer does not transfer application state or ensure that the standby can handle the same requests. Combining DNS and origin-group failover can create layered behavior: the first layer to detect a failure might serve the fallback, while a resolver continues using a previously cached destination.

3. Live static explainer deployment

The existing static-site Terraform root provisions the hosted explainer, not the sample application stack. The S3 bucket blocks public access, enables default encryption and versioning, and expires older object versions after a retention period. A CloudFront Origin Access Control signs access to the private S3 origin. The bucket policy restricts object reads to the distribution. Viewer traffic is redirected to HTTPS.

This deployment provides a real reachable project website, but its presence does not mean the multi-region stack is deployed. The live URL, account resources, and code root must be identified separately in assessment evidence. No claim is made here that the static page's availability proves regional failover.

This separation also reduces the blast radius of educational changes. A reviewer can inspect the larger prototype without replacing the live page that documents it. The two Terraform roots have different resource lifecycles and must be initialized and planned independently. Applying the full-architecture root does not upgrade or import the static-site root.

Bucket versioning and noncurrent-version expiration support recovery from accidental object updates while bounding retained history. They are not a backup strategy for application state, and expiration is not a substitute for verifying object restoration. CloudFront can cache objects independently of the S3 version; an invalidation or suitable cache-control strategy may be needed after updates.

4. Primary network and load balancing

The multi-region root creates a VPC in the primary region, enables DNS support, attaches an internet gateway, and adds two public subnets in distinct available Availability Zones. A public route table associates both subnets. This deliberately small learning topology has no NAT gateways or private application subnets; public-subnet placement is not a recommended default for a production application.

An internet-facing ALB spans those subnets. Its security group permits HTTP ingress from CloudFront's AWS-managed origin-facing prefix list. The application security group accepts HTTP only from the ALB security group. The target group uses `/health` and expects HTTP 200. These restrictions are configuration properties; live reachability and network controls have not been measured in AWS for this stack.

Both subnets are public and instances receive public addresses through the launch path. Security groups restrict inbound application traffic to the ALB, but public-subnet placement and outbound internet access are trade-offs of a compact assignment example. A production design would evaluate private application subnets, controlled egress such as NAT or VPC endpoints, centralized logging, patching, and administrative access. These additions carry cost and operational requirements and are not implied by this code.

The managed CloudFront prefix list avoids maintaining static edge IP ranges in the ALB rule. Its use depends on AWS support and quota behavior in the selected region and account. The VPC input validation reserves sufficient address space for generated subnets. Availability-zone names are account-specific mappings; the configuration chooses two zones returned as available rather than promising identical names across all accounts.

5. Application and scaling

The launch template selects an Amazon Linux 2023 x86_64 image and installs a small threaded Python HTTP service during instance bootstrap. The service returns a page at `/`, health responses at `/health` and `/origin-health`, and 404 for unknown paths. A systemd unit runs it under a dedicated unprivileged account with a restricted capability set and filesystem protections. Instance metadata requires IMDSv2 tokens.

The Auto Scaling group spans both primary subnets and attaches to the ALB target group. Defaults are two minimum, two desired, and six maximum instances. Target tracking uses average CPU with a default 50 percent target. The configuration defines scaling behavior; it does not demonstrate that the group scaled under a measured load or recovered from an instance or Availability Zone failure.

The service is intentionally minimal: it serves no persistent user records, does not authenticate users, and does not depend on a database. This makes the request path easy to inspect, but limits what the system demonstrates. An application with sessions or writes would require an explicit state strategy, data replication or recovery point, consistency model, and dependency failover behavior. A maintenance page is not a second application capable of continuing transactions.

The bootstrap installs Python during instance initialization. First boot therefore depends on repository reachability and may take time before an instance becomes healthy. The health-check grace period and rolling-refresh warm-up settings should be compared with measured startup latency. A live deployment record should include the systemd service status and target registration state rather than relying only on Terraform's apply result.

6. Secondary region and CloudFront behavior

The secondary-region bucket name uses a generated prefix. Terraform configures an S3 website endpoint, encrypts objects at rest, disables ACL access, and allows anonymous `GetObject` only for `index.html`. This public exception is necessary for the website endpoint used in this teaching prototype. The maintenance page must remain non-sensitive. Account-level S3 public-access settings can prevent the policy from being applied.

CloudFront has a primary ALB custom origin and a secondary S3 website custom origin. An origin group selects the maintenance origin for the configured 5xx responses. Viewer requests are redirected to HTTPS, but CloudFront connects to both origins over HTTP in this configuration. The maintenance website endpoint itself also remains directly accessible over HTTP. The `/origin-health*` behavior forwards to the ALB for manual checking; it is not a DNS health check.

CloudFront uses the AWS-generated distribution hostname and default certificate. This avoids a custom name but does not establish a custom-domain identity or Route 53 failure path. CloudFront connects to both origins over HTTP in this prototype. Viewer HTTPS protects the visitor-to-edge connection but not the edge-to-origin leg; a production design must separately determine transport and certificate requirements for each origin.

The maintenance bucket's public-read exception is subject to account-level public-access controls. An account policy that blocks public bucket policies may cause deployment to fail. This is an explicit constraint, not a reason to weaken organization-wide controls without approval. The public endpoint is an assignment trade-off: reviewers can reach the maintenance response, but its content must remain generic and non-sensitive.

7. Monitoring, notification, and security boundaries

CloudWatch observes the ALB `UnHealthyHostCount` metric and publishes an alarm to an SNS topic. An optional email subscription requires recipient confirmation. The alarm definition is present in Terraform; delivery and recovery have not been observed in the target AWS account.

The live static site and the multi-region prototype have different origin security properties. The live site's S3 bucket is private behind OAC. The prototype's maintenance website intentionally permits one public object. Terraform state can contain sensitive resource metadata and must be stored privately. AWS credentials are supplied through an approved identity process, not committed to the repository. The ZIP excludes local Terraform state, private variable files, and `.terraform` caches.

Viewer HTTPS and origin transport are separate security decisions. The live static explainer uses OAC to access a private S3 origin, whereas the multi-region prototype's website endpoint cannot use OAC and requires a public object policy. The latter is a materially different trust boundary. Do not copy its website policy into the private static-site root. For any real deployment, inspect bucket policies, public-access-block settings, security groups, and CloudFront origins in the plan and after apply.

Terraform state can reveal resource names, addresses, configuration values, and metadata. It must not be placed in a public ZIP or repository. Credentials should use the approved AWS identity mechanism and must never be embedded in variable files or screenshots. Review screenshots for account identifiers, email addresses, and console details before submission.

8. Reproducible deployment and teardown procedure

Before deployment, select an authorized account and check quotas, regional availability, estimated charges, and the public maintenance-page exception. Copy `terraform.tfvars.example` to a private `terraform.tfvars`, configure only reviewed values, authenticate, initialize Terraform, run formatting and provider validation, and review the complete plan. Apply only after explicit approval. Record the plan, outputs, account and regions, and timestamps.

The root uses local Terraform state by default. Protect and back up that state or configure an approved remote backend before any shared use. After an approved apply, verify the primary response, health path, origin failure, maintenance response, alarm, optional notification, and restoration according to the runbook. Review the destroy plan before removing resources, then verify the resources are removed and charges have stopped. No such apply or destroy was run for this portfolio revision.

For deployment, verify the active account and region, initialize the correct root, inspect formatting and provider validation, and review a saved plan. Plan review should identify public resources, cross-region resources, expected instance counts, the maintenance bucket policy, SNS subscriptions, and estimated account charges. Applying without checking the target account risks creating billable resources in the wrong environment.

During teardown, inspect the destroy plan and confirm it targets only the test stack. Apply destruction only after evidence has been saved and after checking for retained or independently managed resources. Confirm that the load balancer, distribution, instances, buckets, and alarm resources are removed as expected. Do not delete Terraform state or evidence before destroy completion is verified.

After apply, the included read-only smoke-check script tests the primary page and `/origin-health` endpoint. It requires an HTTPS CloudFront URL and does not induce an outage, exercise the secondary origin, or prove DNS behavior. Use it as a first check only; perform the controlled failure and restoration steps separately, and save their responses and timestamps.

9. Validation results and evidence limits

Project records state that Terraform 1.12.2 formatting and provider-schema validation passed and eight mocked, plan-only Terraform test runs passed. Those tests cover selected defaults, input validation, security-group boundaries, IMDSv2, CloudFront failover responses, maintenance-bucket access, and alarm configuration. Mocked provider tests create no AWS resources.

Terraform was unavailable in the environment used to rebuild this submission; the checks above were not rerun during this revision. No current AWS plan, apply, runtime test, failover response, alarm delivery, scaling result, or recovery measurement is included. The browser check of the live CloudFront URL verifies only that the static explainer loaded at the time observed. Detailed evidence status and the outstanding collection checklist are in the validation register.

These evidence categories must remain separate in assessment claims:

- **Configuration evidence:** source and mocked plans demonstrate intended

resource arguments and validation rules.

- **Service evidence:** provider validation and local checks show that a

selected toolchain accepts the configuration; they do not prove cloud permissions or resource creation.

- **Deployment evidence:** a reviewed plan and successful apply identify

resources created in a specific account and regions.

- **Runtime evidence:** timestamped requests, alarm history, and recovery

observations show deployed behavior during a controlled exercise.

The portfolio has configuration evidence and a reachable static explainer, but not current deployment or runtime evidence for the multi-region stack. The historical Phase 2 report describes earlier activity, but its source, state, and logs are absent; it is not substituted for current proof. Future evidence should include commands, tool versions, results, timestamps, expected and observed outputs, and limitations.

10. DNS failover design, reflection, and next steps

The Terraform root has no Route 53 records, health checks, custom domain, or ACM certificate. A console URL for an S3 bucket is not a controlled DNS name or hosted-zone identifier. DNS cannot be added responsibly with placeholder values or without deciding the health signal, HTTPS behavior, record failover policy, TTL, and rollback approach.

A future candidate is Route 53 failover records for a controlled hostname pointing to primary and standby CloudFront distributions. Both targets would need compatible HTTPS certificates, and the primary health check must observe the real application rather than a static edge response. The design must account for DNS caching and the separate CloudFront origin-group behavior. This candidate has not been implemented or tested.

DNS record design should identify the record name and type, routing policy, target, alias health-evaluation setting, TTL behavior, and primary and standby health signals. The health-check endpoint must be publicly reachable by the monitoring system and should represent application health, not merely the fact that DNS or a CDN edge responds. Failure thresholds introduce detection-delay and false-positive trade-offs. Standby health should also be monitored so failover does not send users to an unavailable target.

The test plan should distinguish DNS answer changes from application recovery. Record resolution through multiple recursive resolvers, client-visible HTTPS responses, health-check state transitions, and recovery after primary restoration. Define rollback before testing and avoid changing a shared production record. Domain ownership, certificate renewal, public health-check costs, and duplicate distributions belong in the operating plan.

The DNS extension should be added as a separately reviewed Terraform change with optional inputs that must be supplied only after these choices are settled. It should not silently reuse the existing CloudFront distribution certificate or S3 website endpoint as though they support arbitrary custom-domain HTTPS. Any DNS testing should use a dedicated subdomain and test records, an approved change window, and a rollback route that does not depend on the failure being tested.

Finalisation has improved consistency between the source, PDFs, and ZIP; made the distinction between live hosting, prototype code, and proposed DNS behavior explicit; and documented a safe operational evidence plan. The largest remaining improvement is authorized deployment and controlled runtime testing, followed by a DNS design using actual domain and hosted-zone details. Until those steps are completed, the accurate outcome is a live static explainer plus reviewable infrastructure code, not a proven end-to-end failover service.

The next revision should update the conception diagram to distinguish the implemented CloudFront origin path from the proposed DNS layer, then add DNS resources only after domain, hosted-zone, certificate, health, and cost decisions are approved. After controlled deployment, publish redacted, timestamped evidence and revise the abstract to match actual observations. Tutor feedback should be incorporated when received; none is invented or implied in this report.